Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Data Mining

Lab - 2

¥@$# Kakadiya | 23010101121 | 11/6/2025

Numpy & Perform Data Exploration with Pandas

Numpy

1. NumPy (Numerical Python) is a powerful open-source library in Python used for numerical and scientific computing.
2. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them efficiently.
3. NumPy is highly optimized and written in C, making it much faster than using regular Python lists for numerical operations.
4. It serves as the foundation for many other Python libraries in data science and machine learning, like pandas, TensorFlow, and scikit-learn.
5. With features like broadcasting, vectorization, and integration with C/C++ code, NumPy allows for cleaner and faster code in numerical computations.

Step 1. Import the Numpy library

In [1]:
```
import numpy as np

np.__version__
```

Out[1]:
```
'1.26.4'
```

Step 2. Create a 1D array of numbers

In [2]:
```
arr = np.array([1,2,3,4,5,6,7])
arr
```

Out[2]:
```
array([1, 2, 3, 4, 5, 6, 7])
```

In [3]:
```
arr=np.arange(1,8,2)
arr
```

Out[3]:
```
array([1, 3, 5, 7])
```

In [4]:
```
arr2d=np.array([[1,2],[3,4],[5,6]])
arr2d
```

Out[4]:
```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

Step 3. Reshape 1D to 2D Array

In [5]:
```
arrReshape=np.arange(8).reshape(2,4)

print(arrReshape)
print(arrReshape.dtype)
print(arrReshape.size)
print(type(arrReshape))
print(type(arrReshape[0,1]))
print(arrReshape.ndim)
```

```
[[0 1 2 3]
 [4 5 6 7]]
int32
8
<class 'numpy.ndarray'>
<class 'numpy.int32'>
2
```

Step 4. Create a Linspace array

In [6]:
```
arr=np.linspace(0,10,5)
arr
```

Out[6]:
```
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

Step 5. Create a Random Numbered Array

In [7]:
```
arr=np.random.rand(7)
arr
```

Out[7]:
```
array([0.89131114, 0.52252606, 0.70993858, 0.77093598, 0.35612336,
       0.52889868, 0.18022875])
```

Step 6. Create a Random Integer Array

In [8]:
```
arr=np.random.randint(1,7,size=7)
arr
```

Out[8]:
```
array([4, 4, 3, 3, 4, 3, 5])
```

Step 7. Create a 1D Array and get Max,Min,ArgMax,ArgMin

In [9]:
```
arr=np.arr=np.random.randint(1,700,size=7)
arr
```

Out[9]:
```
array([ 93, 504, 329, 280, 625, 658, 120])
```

In [10]:
```
arr.max()
```

Out[10]:
```
658
```

In [11]:
```
arr.min()
```

Out[11]: 93

In [12]:
```
arr.argmax()
```

Out[12]: 5

In [13]:
```
arr.argmin()
```

Out[13]: 0

In [14]:
```
arr.mean()
```

Out[14]: 372.7142857142857

Step 8. Indexing in 1D Array

In [15]:
```
arr=np.arr=np.random.randint(1,700,size=7)
print(arr)
print(arr[6])
print(arr[-1])
```

```
[391 421 217 331 687 574 176]
176
176
```

Step 9. Indexing in 2D Array

In [16]:
```
arr2d=np.array([[1,2],[3,4],[5,6]])
print(arr2d[1,0])
print(arr2d[-1,0])
```

```
3
5
```

Step 10. Conditional Selection

In [17]:
```
arr=np.arr=np.random.randint(1,700,size=7)
print(arr)
arr[arr>3]
```

```
[681 451 102 646 394 644 353]
```

Out[17]: array([681, 451, 102, 646, 394, 644, 353])

In [18]:
```
arr2d=np.array([[1,2],[3,4],[5,6]])
print(arr2d)
```

```
[[1 2]
 [3 4]
 [5 6]]
```

🔥 You did it! 10 exercises down — you're on fire! 🔥

Pandas

Step 1. Import the necessary libraries

In [19]:
```
import pandas as pd
pd.__version__
```

Out[19]: '2.2.2'

Step 2. Import the dataset from this address.

In [20]:
```
df=pd.read_csv('https://raw.githubusercontent.com/justmarkham/DAT8/master/data/u.user',sep='|')
df
```

Out[20]:

	user_id	age	gender	occupation	zip_code
0	1	24	M	technician	85711
1	2	53	F	other	94043
2	3	23	M	writer	32067
3	4	24	M	technician	43537
4	5	33	F	other	15213
...	...	...	...	...	...
938	939	26	F	student	33319
939	940	32	M	administrator	02215
940	941	20	M	student	97229
941	942	48	F	librarian	78209
942	943	22	M	student	77841

943 rows × 5 columns

Step 3. Assign it to a variable called users and use the 'user_id' as index

In [21]:
```
users=df.set_index('user_id')
users
```

Out[21]:

user_id	age	gender	occupation	zip_code
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
...	...	...	...	...
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

943 rows × 4 columns

Step 4. See the first 25 entries

In [22]:
```
users.head(25)
```

Out[22]:

user_id	age	gender	occupation	zip_code
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
6	42	M	executive	98101
7	57	M	administrator	91344
8	36	M	administrator	05201
9	29	M	student	01002
10	53	M	lawyer	90703
11	39	F	other	30329
12	28	F	other	06405
13	47	M	educator	29206
14	45	M	scientist	55106
15	49	F	educator	97301
16	21	M	entertainment	10309
17	30	M	programmer	06355
18	35	F	other	37212
19	40	M	librarian	02138
20	42	F	homemaker	95660
21	26	M	writer	30068
22	25	M	writer	40206
23	30	F	artist	48197
24	21	F	artist	94533
25	39	M	engineer	55107

Step 5. See the last 10 entries

In [23]:
```
users.tail(10)
```

Out[23]:

user_id	age	gender	occupation	zip_code
934	61	M	engineer	22902
935	42	M	doctor	66221
936	24	M	other	32789
937	48	M	educator	98072
938	38	F	technician	55038
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

Step 6. What is the number of observations in the dataset?

In [24]:
```
users.shape[0]
```

Out[24]:
```
943
```

Step 7. What is the number of columns in the dataset?

In [25]:
```
users.shape[1]
```

Out[25]:
```
4
```

In [26]:
```
users.shape
```

Out[26]:
```
(943, 4)
```

Step 8. Print the name of all the columns.

In [27]:
```
users.columns
```

Out[27]:
```
Index(['age', 'gender', 'occupation', 'zip_code'], dtype='object')
```

Step 9. How is the dataset indexed?

In [28]:
```
users.index
```

Out[28]:
```
Index([  1,   2,   3,   4,   5,   6,   7,   8,   9,  10,
       ...
       934, 935, 936, 937, 938, 939, 940, 941, 942, 943],
      dtype='int64', name='user_id', length=943)
```

In [29]:
```
users.index.name
```

Out[29]:
```
'user_id'
```

Step 10. What is the data type of each column?

In [30]:
```
users.dtypes
```

Out[30]:
```
age            int64
gender        object
occupation    object
zip_code      object
dtype: object
```

Step 11. Print only the occupation column

In [31]:
```
users['occupation']
```

Out[31]:
```
user_id
1         technician
2              other
3             writer
4         technician
5              other
           ...
939          student
940    administrator
941          student
942        librarian
943          student
Name: occupation, Length: 943, dtype: object
```

Step 12. How many different occupations are in this dataset?

In [32]:
```
users['occupation'].nunique()
```

Out[32]:
```
21
```

In [33]:
```
users['occupation'].unique()
```

Out[33]:
```
array(['technician', 'other', 'writer', 'executive', 'administrator',
       'student', 'lawyer', 'educator', 'scientist', 'entertainment',
       'programmer', 'librarian', 'homemaker', 'artist', 'engineer',
       'marketing', 'none', 'healthcare', 'retired', 'salesman', 'doctor'],
      dtype=object)
```

Step 13. What is the most frequent occupation?

In [34]:
```
users['occupation'].value_counts().head(1)
```

Out[34]:
```
occupation
student    196
Name: count, dtype: int64
```

Step 14. Summarize the DataFrame.

In [35]:
```
users.describe()
```

Out[35]:

	age
count	943.000000
mean	34.051962
std	12.192740
min	7.000000
25%	25.000000
50%	31.000000
75%	43.000000
max	73.000000

Step 15. Summarize all the columns

In [36]:
```
users.describe(include='all')
```

Out[36]:

	age	gender	occupation	zip_code
count	943.000000	943	943	943
unique	NaN	2	21	795
top	NaN	M	student	55414
freq	NaN	670	196	9
mean	34.051962	NaN	NaN	NaN
std	12.192740	NaN	NaN	NaN
min	7.000000	NaN	NaN	NaN
25%	25.000000	NaN	NaN	NaN
50%	31.000000	NaN	NaN	NaN
75%	43.000000	NaN	NaN	NaN
max	73.000000	NaN	NaN	NaN

Step 16. Summarize only the occupation column

In [37]:
```
users['occupation'].describe()
```

Out[37]:
```
count         943
unique         21
top       student
freq          196
Name: occupation, dtype: object
```

Step 17. What is the mean age of users?

In [38]:
```
users['age'].mean()
```

Out[38]:
```
34.05196182396607
```

Step 18. What is the age with least occurrence?

In [39]:
```
users['age'].value_counts().tail()
```

Out[39]:
```
age
7     1
66    1
11    1
10    1
73    1
Name: count, dtype: int64
```

You're not just learning, you're mastering it. Keep aiming higher! 🚀